# TP7 Archi-L2: XV6 File System

[Durée: 6 heures]

Omar Rifki - omar.rifki@univ-littoral.fr
modifié du TP de Eric Ramat

12 février 2024

## 0 Objectif

Ce TP a pour objectif d'analyser le système de fichiers de XV6. Cette analyse se fera à partir du fichier image (`fs.img`) généré par le projet XV6 version riscv (https://github.com/mit-pdos/xv6-riscv).

## 1 Lecture des fichiers

Dans la version de XV6 utilisée, tous les blocs possèdent une taille de 1024 octets. Voici une description de la structure du fichier image :

```
/*
 *  0     1     2             i             m             d         N-1
 * +----+----+----...----+----...----+----...----+----...----+
 * | BB | SB |     LB      |     IB      |     MB      |     DB      |
 * +----+----+----...----+----...----+----...----+----...----+
 *            |<-- Nl --->|<-- Ni --->|<-- Nm --->|<-- Nd --->|
 */
```

L'adresse de chaque zone est décrite dans le super-bloc qui se trouve dans le deuxième bloc.

Dans un premier temps, nous allons écrire une fonction en C/C++ afin de retrouver les éléments définissants les différentes zones (Boot block, Super block, Log block, Inode block, Bitmap block et Data block) du système d'exploitation.

**Question 1.** Ecrire une fonction C/C++ `display_info(...)` qui affiche le nombre de blocs de chaque zone. La fonction de lecture du fichier image est donnée.

Pour vous aider, trois types sont déclarés :
- block : notion générique de bloc de 1024 octets
- superblock : struct avec les différents attributs du super-bloc
- dinode : struct avec les différents attributs d'un inode

Voici quelques indications supplémentaires sur les valeurs à trouver :

```
/*
 * BSIZE = 1024
 * IPB = BSIZE / sizeof(struct dinode) = 1024 / 64 = 16
 *
 * BB: boot block     [0, 0]
 * SB: super block    [1, 1]
 * LB: log block      [l, l + Nl - 1] (l = sb.logstart)
 * IB: inode block    [i, i + Ni - 1] (i = sb.inodestart)
 * MB: bitmap blocks  [m, m + Nm - 1] (m = sb.bmapstart)
 * DB: data blocks    [d, d + Nd - 1] (d = Nb + Ns + Nl + Nm + Nb)
 * N  = sb.size = Nb + Ns + Nl + Ni + Nm + Nd
 * Nb = 1
 * Ns = 1
 * Nl = sb.nlog
 * Ni = sb.ninodes / IPB + 1
 * Nm = N / (BSIZE * 8) + 1
 * Nd = sb.nblocks
 *
 * BB: boot block    [0,  0]      = [0x00000000, 0x000003ff]
 * SB: super block   [1,  1]      = [0x00000400, 0x000007ff]
 * LB: log block     [2,  31]     = [0x00000800, 0x00007fff]
 * IB: inode block   [32, 44]     = [0x00008000, 0x0000b3ff]
 * BB: bitmap block  [45, 45]     = [0x0000b400, 0x0000b7ff]
 * DB: data block    [46, 999]    = [0x0000b800, 0x000f9fff]
 *
 * N = 1000
 * Nl = 30
 * Ni = 200 / 16 + 1 = 13
 * Nm = 1000 / (512 * 8) + 1 = 1
 * Nd = 1000 - (1 + 1 + 30 + 13 + 1) = 954
 */
```

### Astuce pour convertir un block en superblock

```
block b;
superblock* sb = (superblock*)(&b);
// sb->size ou (*sb).size : block number
```

**Question 2.** Ecrire une fonction C/C++ `display_inode(...)` qui affiche les caractéristiques des premier inodes (`block number` = 32).

L'attribut `type` d'un inode permet de savoir si c'est un inode pour un répertoire (1), un fichier (2) ou un `device` (3). Si le type est égal à 0, alors c'est un inode non utilisé.

Voici un exemple de sortie console possible :

```
Inode 1: type = DIR ; size = 1024 bytes ; links = 1
Inode 2: type = FILE ; size = 2059 bytes ; links = 1
Inode 3: type = FILE ; size = 23528 bytes ; links = 1
Inode 4: type = FILE ; size = 22360 bytes ; links = 1
Inode 5: type = FILE ; size = 12768 bytes ; links = 1
Inode 6: type = FILE ; size = 26848 bytes ; links = 1
Inode 7: type = FILE ; size = 23480 bytes ; links = 1
Inode 8: type = FILE ; size = 22344 bytes ; links = 1
Inode 9: type = FILE ; size = 22312 bytes ; links = 1
```

```
Inode 10: type = FILE ; size = 25744 bytes ; links = 1
Inode 11: type = FILE ; size = 22440 bytes ; links = 1
Inode 12: type = FILE ; size = 22424 bytes ; links = 1
Inode 13: type = FILE ; size = 41312 bytes ; links = 1
Inode 14: type = FILE ; size = 23368 bytes ; links = 1
Inode 15: type = FILE ; size = 150648 bytes ; links = 1
Inode 16: type = FILE ; size = 37640 bytes ; links = 1
Inode 17: type = FILE ; size = 24608 bytes ; links = 1
Inode 18: type = FILE ; size = 21840 bytes ; links = 1
```

**Question 3.** Ecrire une fonction C/C++ `display_dir(image& img, uint inumber)` qui affiche la liste des fichiers contenu dans un répertoire où le paramètre `inumber` est le numéro de l'inode. Cette fonction doit réaliser le même travail que la commande `ls` mais à partir des informations contenues dans les blocs `data` liés à l'inode de type `Directory`.

Dans notre exemple, il y a qu'un seul répertoire dans le `file system` et le numéro de l'inode correspondant est 1 (voir étape 2).

**Indication :** dans un `data block`, une entrée de repértoire est stockée sous la forme de deux attributs : inum pour le numéro d'inode et un tableau de 13 caractères max pour le nom du fichier.

Voici une sortie console possible :

```
[ 1 ; . ]
[ 1 ; .. ]
[ 2 ; README ]
[ 3 ; cat ]
[ 4 ; echo ]
[ 5 ; forktest ]
[ 6 ; grep ]
[ 7 ; init ]
[ 8 ; kill ]
[ 9 ; ln ]
[ 10 ; ls ]
[ 11 ; mkdir ]
[ 12 ; rm ]
[ 13 ; sh ]
[ 14 ; stressfs ]
[ 15 ; usertests ]
[ 16 ; grind ]
[ 17 ; wc ]
[ 18 ; zombie ]
```

**Question 4.** Expliquer pourquoi le chemin ".." a pour numéro d'inode le même que ".".

**Question 5.** Développer une fonction C/C++ `display_text_file` qui affiche le contenu du fichier README. Cette fonction prendra en paramètre l'image et le nom du fichier :

```
void display_text_file(image& img, const std::string& file_name);
```

Il faut donc partir du répertoire en inode 1, puis rechercher l'entrée dans le répertoire qui correspond au fichier README, cela nous permet de connaître son numéro d'inode. A partir du numéro d'inode,

on accède à l'inode et donc à la liste des adresses des `data blocks` qui composent le fichier texte. Indication : une adresse est valide, si elle est différente de $0$.

## 2 Suppression et création de fichiers

Dans cette section, nous aborderons les opérations de suppression et création de fichiers et de répertoires.

**Question 6.** Ecrire une fonction C/C++ `delete_file(image& img, const std::string& file_name)` qui a pour objectif la suppression d'un fichier du système de fichiers. Supprimer un fichier consiste à libérer le bloc d'inode utilisé par le fichier et à libérer les blocs de données. Attention, il faudra mettre à jour le bitmap des blocs libres.

Il faut décomposer la fonction en quatre parties :

- recherche de l'inode du fichier à supprimer (on pourra réutiliser une partie de la fonction `display_text_file(...)` du TP précédent)
- mettre à $0$ l'attribut `type` de l'inode du fichier
- mettre à $0$ les bits correspondants aux blocs de données utilisés par le fichier
- retirer dans le répertoire l'entrée correspondant au fichier

On vérifiera que le fichier n'apparaît plus en appelant les fonctions `display_dir(...)` et `display_inode(...)`. De plus, on vérifiera que le tableau bitmap est mis à jour. Voici le code d'une nouvelle fonction qui va nous permettre de le vérifier.

```
void display_free_data_block(image &img)
{
  superblock *p = (superblock *) (&img.data[1]);
  uint8_t *q = (uint8_t *) (&img.data[p->bmapstart]);
  uint c1 = 0, c2 = 0;
  uint8_t mask = 1;
  uint m = 0, k = 0;

  for (uint i = 0; i < p->nblocks; ++i) {
    if (!(q[m] & mask)) {
      ++c1;
    } else {
      ++c2;
    }
    ++k;
    if (k == 8) {
      k = 0;
      mask = 1;
      ++m;
    } else {
      mask <<= 1;
    }
  }
  std::cout << "Free block number = " << c1 << std::endl;
  std::cout << "No free block number = " << c2 << std::endl;
  std::cout << "Total block number = " << p->nblocks << std::endl;
}
```

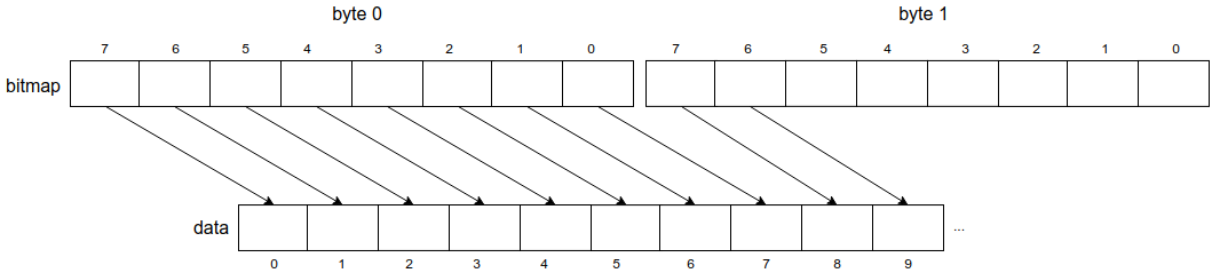
Avant la suppression du fichier, on a :

```
Free block number = 370
No free block number = 584
Total block number = 954
```

Après,

```
Free block number = 373
No free block number = 581
Total block number = 954
```

**Rappel et Indication :** dans le bloc bitmap, un bit à $0$ indique que le bloc de données est libre. **Le bit du premier `data block` est le 8ème bit du premier octet du bloc bitmap.** Le bloc bitmap contient 1024 octets soit 8192 bits. Pour manipuler ce tableau de bits, on utilisera le type C++ `uint8_t`. Ce type représente un entier non signé sur 8 bits.




**Astuce :** utiliser des masques binaires et les opérateurs binaires (& : et binaire, | ou binaire et ~ non binaire).

## ********** BONUS **********

**Question 7.** Ecrire une fonction `new_directory(image& img, const std::string& directory_name)` pour la création d'un nouveau répertoire. Pour cela, il est nécessaire de réserver un bloc d'inode. Le bloc sélectionné sera le premier que l'on trouve avec un type égal à $0$. Ensuite, il faut réserver un bloc de données. Le bloc bitmap permet de le trouver. On prendra une nouvelle fois, le premier trouvé.

Comme c'est un nouveau répertoire, il faut définir seulement deux entrées dans le répertoire : "." et "..".

**Question 8.** Pour cette dernière étape, nous allons écrire un fonction C/C++ `new_file(image& img, const std::string& file_name, char* buffer, uint size)` qui va nous permettre d'enregistrer un nouveau fichier. Pour faire simple, il faudra définir un tableau d'au moins 2100 caractères avec le contenu que vous voulez. Comme pour les répertoires, il faut réserver un bloc d'inode, initialiser les attributs, réserver les bloc de données et stocker le tableau de caractères dans les bloc.

On pourra vérifier le bon fonctionnement de la fonction en faisant une relecture du fichier via la fonction écrite dans le TP précédent.